COMP6010 Assignment: From Bayesian Inference to Generative AI

Planning and Handling Uncertainty in Machine Learning

Satria Pamungkas • Student ID: **21522611** • Part C option: **C2** (animal drawings, flow matching)

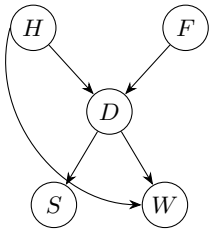
Supplement: `Satria_Pamungkas_21522611_COMP6010_Supplement.zip`

Introduction. This report answers three connected questions with one method: probability. Part A performs *inference* in a fully known five-variable Bayesian network, where the difficulty is conditioning on evidence correctly. Part B *learns* a single unknown parameter and shows why a decision must be driven by the whole posterior rather than a point estimate. Part C *learns a generative model* — a conditional flow-matching network trained from scratch on Quick, Draw! animal drawings — and shows that the same factorise/learn/sample pattern scales from five binary variables to images. Every number cited is produced by the notebooks in the supplement and stored in a named CSV or JSON file; file names appear in `grey monospace`.

Part A: Bayesian-network simulation and sampling inference

A1. Model, factorisation and exact reference values

The five binary variables are H (hot weather), F (irrigation fault), D (dry soil), S (sensor reports "dry"), W (plants wilting), with edges $H \to D$, $F \to D$, $D \to S$, $D \to W$, $H \to W$ and independent roots $P(H{=}1) = 0.30$, $P(F{=}1) = 0.08$.



The graph gives the factorisation

$$P(H, F, D, S, W) = P(H)\,P(F)\,P(D \mid H, F)\,P(S \mid D)\,P(W \mid H, D),$$

i.e. 2+2+4+2+4 CPT entries rather than the 31 free parameters of an unconstrained joint. One implied conditional independence is $S \perp W \mid D, H$: S depends on the network only through its single parent D, so once D (and H, which also parents W) is fixed, the sensor reading says nothing further about wilting. Verified numerically to $\max |P(S \mid D, H, W) - P(S \mid D, H)| < 10^{-16}$.

Enumerating all $2^5 = 32$ states gives $\sum P = 1$ to within 2×10^{-16} and the exact references in Table 1. Two further checks matter. First, marginalising recovers $P(H{=}1) = 0.30$ and $P(F{=}1) = 0.08$, and $P(H{=}1, F{=}1) = P(H{=}1)P(F{=}1)$ confirms prior root independence. Second, and more usefully, I deliberately transposed the H, F indices in the D lookup: the joint *still* summed to 1, so a normalisation test alone would not have caught the bug, whereas the query values changed visibly (q_1 became 0.2044). A passing normalisation check is necessary but far from sufficient. `A1_exact_reference_values.csv`, `A1_joint_enumeration_32_states.csv`

Table 1: Exact values by enumeration, with the evidence probability that governs rejection-sampling efficiency.

Query	Definition	Exact value	P(evidence)	Retained per 1,000
q_1	$P(F{=}1 \mid S{=}1)$	0.231504	0.26816	268
q_2	$P(F{=}1 \mid S{=}1, W{=}1)$	0.293294	0.15932	159
q_3	$P(F{=}1 \mid S{=}1, W{=}1, H{=}1)$	0.168675	0.10956	110

A2. Forward sampling and rejection-based inference

To sample a binary X with $P(X{=}1 \mid \text{pa}) = p$, draw $u \sim \text{Uniform}[0, 1)$ and set $x = \mathbb{1}[u < p]$; since the uniform CDF is $F(u) = u$, we get $P(x{=}1) = P(u < p) = p$ exactly. This is inverse-CDF sampling with the unit interval cut at p, so the two sub-interval widths *are* the two probabilities. The correct CPT row is selected using the already-sampled parent values, which is why the loop must follow the topological order (H, F, D, S, W): every variable's parents are assigned before it is drawn, and the resulting assignments are i.i.d. draws from the exact joint.

Rejection sampling then discards every sample inconsistent with the evidence e and reports the fraction of survivors with $F{=}1$; this is valid because conditioning on an event and restricting to it are the same operation. Its cost is that the retained fraction is $P(e)$. I generate one joint sample set per seed and reuse it for all three queries, at $N \in \{10^3, 10^4, 10^5\}$ with five seeds each. Measured retained fractions match Table 1 closely (e.g. 0.1590 against 0.15932 for q_2 at $N{=}10^5$). No estimate was undefined across the 45 runs — the smallest retained count was 107, for q_3 at $N{=}10^3$ — but the 0/0 case returns `NaN` rather than an invented value, as required. `A2_rejection_per_run.csv`

A3. Likelihood weighting

I implement likelihood weighting only (the brief warns against adding Gibbs merely to increase code volume). Evidence variables are *clamped*; the rest are sampled in topological order; each sample is weighted by $w = \prod_{E_i \in \mathcal{E}} P(E_i{=}e_i \mid \text{pa}(E_i))$, and

$$\hat{P}(F{=}1 \mid e) = \frac{\sum_{k:F^{(k)}=1} w^{(k)}}{\sum_k w^{(k)}}.$$

Why weights are required. Clamping draws from the mutilated network $Q(x_{-\mathcal{E}}) = \prod_{X_i \notin \mathcal{E}} P(x_i \mid \text{pa}_i)$, not from the posterior. Since $P(x_{-\mathcal{E}} \mid e) = Q(x_{-\mathcal{E}})\,w(x_{-\mathcal{E}})/P(e)$, the weight is exactly the importance ratio, giving $\mathbb{E}_Q[w\mathbb{1}(F{=}1)] = P(F{=}1, e)$ and $\mathbb{E}_Q[w] = P(e)$. Counting *unweighted* would compute $Q(F{=}1)$, which here is just the prior: I measured 0.0800 against the exact $q_2 = 0.2933$, because clamping a *descendant* cannot change how its

ancestors were drawn. The weights are what carry evidence back up the graph. Normalisation must use $\sum_k w^{(k)}$ (a Monte-Carlo estimate of $P(e)$; the mean weight came to 0.1600 against the exact 0.159316) — dividing by N would return the joint $P(F{=}1, e)$ instead. Note H in q_3 is a *root* evidence variable, so it contributes the constant 0.30; only non-root evidence makes weights vary. `A3_likelihood_weighting_per_run.csv`

A4. Comparison and interpretation

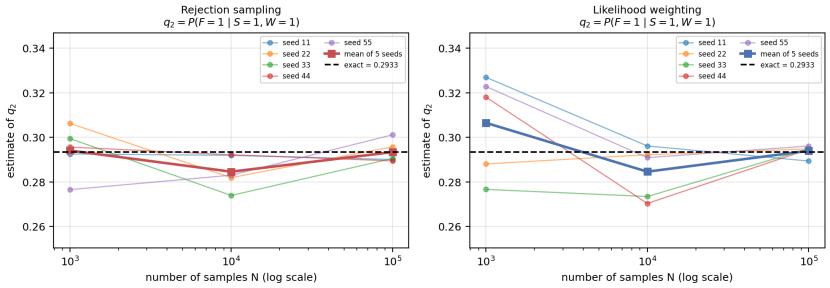


Table 2: Mean ± sd over five seeds.

	N	Rejection	Lik. weight.
q_1	10^3	.238±.014	.251±.040
	10^4	.228±.007	.227±.009
	10^5	.231±.003	.231±.002
q_2	10^3	.294±.011	.307±.023
	10^4	.285±.008	.285±.012
	10^5	.293±.005	.294±.003
q_3	10^3	.170±.019	.173±.016
	10^4	.161±.018	.167±.004
	10^5	.169±.005	.169±.001

Figure 1: Convergence of q_2 (five seeds, exact value dashed). Both estimators are consistent; spread narrows as $N^{-1/2}$. `A4_convergence_q2.png`, `A4_summary_table.csv`

Evidence accumulation. Against the prior $P(F{=}1) = 0.08$: a dry-soil alert raises the fault probability to $q_1 = 0.2315$ (2.9× the prior) but leaves it low, because the sensor is only 90% accurate and dry soil has a far more common competing cause (hot weather, prior 0.30 versus the fault's 0.08). Adding wilting gives $q_2 = 0.2933$: W is a *second, independent* symptom of dry soil, so it corroborates rather than repeats the sensor — $S \perp W \mid D, H$ means they are two separate measurements of the same hidden state. Adding hot weather *reverses* the direction, $q_3 = 0.1687$. This is **explaining away**: D is a collider on $H \to D \leftarrow F$, so confirming an alternative sufficient cause makes the fault less necessary. Practically, a dry-soil alert on a hot day is much weaker evidence of a fault than the same alert on a cool day. These are conditional probabilities under the supplied hypothetical model, not evidence of real-world causation. `A4_evidence_sequence.csv`

Accuracy versus effort. Rejection sampling keeps only the fraction $P(e)$ of its samples, so each added evidence variable shrinks the usable set: at $N{=}10^5$ it retains ∼26,800 samples for q_1 but only ∼10,900 for q_3. Likelihood weighting retains all N. With the five seeds the brief requires, this ordering is *not* reliably visible — rejection happens to win some q_1/q_2 rows in Table 2 — so I ran a supplementary 400-seed check: likelihood weighting then has lower sd and lower MAE in **9 of 9** (query, N) combinations, with a sd ratio of 1.04–1.26 for q_1/q_2 and a consistent 2.05× for q_3, the most heavily conditioned query. That five seeds cannot rank two estimators whose variances differ by less than a factor of two is itself the lesson. On weight concentration, the Kish effective sample size $\text{ESS} = (\sum w)^2 / \sum w^2$ gives ESS/N of 0.40, 0.22 and 0.46 for q_1, q_2, q_3: the weight takes only a few distinct values here (discrete evidence CPT entries), the top 1% of weights hold under 6% of total mass, so *no* few samples dominate. That is a property of this small network with mild evidence, not a general guarantee. One caveat on cost: rejection generates one sample set reused by all three queries, whereas likelihood weighting must re-run per evidence set, so I report their timings separately rather than equating a "sample" across methods. `A4_variance_check_400seeds.csv`, `A4_timing.csv`

Why more samples need not help every run. Each estimate is a random variable; increasing N reduces its *variance*, so error falls on average and the seed spread narrows, but a single run can be lucky at small N. Concretely, for q_2 under rejection sampling seeds 11 and 33 got *worse* from $N{=}10^3$ to 10^4, and seeds 11 and 44 from 10^4 to 10^5. Only the aggregated mean and sd are evidence about an estimator; a single run is not, which is exactly why five seeds are required.

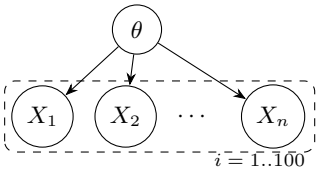
Why fixing downstream evidence is invalid. Forward sampling draws each variable from $P(x_i \mid \text{pa}_i)$, so influence flows only parent → child. Overwriting an observed descendant after the fact cannot change how its ancestors were drawn: doing this for $S{=}1, W{=}1$ over 200,000 samples returned 0.0799 — the prior — instead of $q_2 = 0.2933$. Diagnostic reasoning runs *against* the arrows, so a correction is mandatory: either discard mismatching samples (rejection) or re-weight by the evidence likelihood (likelihood weighting).

Part B: Bayesian learning with a Beta prior

B1. Formulation

Each grasp attempt succeeds or fails, so the Bernoulli distribution — the only distribution on $\{0, 1\}$, with one parameter — is the natural likelihood, $P(X_i{=}x \mid \theta) = \theta^x(1-\theta)^{1-x}$. For n conditionally independent attempts with s successes the likelihood is $\theta^s(1-\theta)^{n-s}$, depending on the data only through the sufficient statistics (s, n). The parameter θ is not an outcome: it is an unknown *continuous* quantity in $(0, 1)$, so uncertainty about it needs a density, and the Beta family is supported exactly on $(0, 1)$. The decisive property is **conjugacy**: the Beta kernel $\theta^{\alpha-1}(1-\theta)^{\beta-1}$ has the same algebraic form as the likelihood, so the posterior is again Beta and updating is exact

arithmetic on two numbers. Here α, β are *prior shape parameters, not data*; they behave like $\alpha-1$ pseudo-successes and $\beta-1$ pseudo-failures, so Beta$(2,3)$ is a mildly pessimistic belief (mean 0.4) carrying about as much weight as three real attempts.



θ is the single shared parent of every observation, so $f(\theta, x_1..x_n) = f(\theta)\prod_i P(x_i \mid \theta)$ and $X_i \perp X_j \mid \theta$: once θ is fixed, attempt 3's outcome says nothing about attempt 4. The X_i are *not* marginally independent — integrating θ out does not factorise. I verified this by simulation: $P(X_2{=}1 \mid X_1{=}1) = 0.4283$ versus $P(X_2{=}1 \mid X_1{=}0) = 0.3335$ under the prior, but both equal 0.55 when θ is held fixed. This common-cause dependence is what *makes learning possible*: if attempts were marginally independent, data could not inform us about θ. Note also that a density is not a probability — $f(0.8)$ may exceed 1, while $P(\theta{=}0.8) = 0$ exactly, which is why B3 asks for a tail mass rather than a density value.

B2. Posterior updating

Substituting the Bernoulli likelihood and Beta prior into Bayes' rule and absorbing every θ-free factor into the normalising constant,

$$f(\theta \mid \mathcal{D}) \;\propto\; \underbrace{\theta^{s}(1-\theta)^{n-s}}_{\text{likelihood}} \cdot \underbrace{\theta^{\alpha-1}(1-\theta)^{\beta-1}}_{\text{prior}} = \theta^{(\alpha+s)-1}(1-\theta)^{(\beta+n-s)-1} \implies \theta \mid \mathcal{D} \sim \text{Beta}(\alpha+s,\ \beta+n-s),$$

so updating adds successes to α and failures to β. Sequential and pooled updating *must* agree, because the likelihood factorises across batches and multiplication is order-independent; both give Beta(2+85, 3+15) = **Beta(87, 18)** and the notebook asserts the equality. I also checked the conjugate shortcut against brute-force numerical Bayes on a 200,001-point grid (max density difference $\sim 10^{-9}$) and confirmed all six batch orderings return the same posterior. `B2_sequential_updates.csv`

Table 3: Cumulative point estimates. The three estimators converge as the prior's weight (∼3 pseudo-attempts) is swamped by data. `B2_point_estimates.csv`

Stage	Posterior	Cum. n	Cum. s	ML s/n	Post. mean	MAP
Prior	Beta$(2,3)$	0	0	—	0.4000	0.3333
After batch 1	Beta$(8,7)$	10	6	0.6000	0.5333	0.5385
After batch 2	Beta$(24,11)$	30	22	0.7333	0.6857	0.6970
After batch 3	Beta$(87,18)$	100	85	0.8500	**0.8286**	0.8350

The gap |post. mean − ML| shrinks from 0.067 after batch 1 to 0.021 after batch 3: the prior pulls hardest when data are scarce. MAP sits between ML and the posterior mean because the mode of this left-skewed posterior lies above its mean.

B3. Uncertainty and the deployment decision

Three distinct quantities, all from Beta$(87,18)$:

(1) Posterior predictive. We do not know θ, so we average over the posterior rather than plugging in an estimate: $P(X_{n+1}{=}1 \mid \mathcal{D}) = \int_0^1 \theta\, f(\theta \mid \mathcal{D})\, d\theta = \mathbb{E}[\theta \mid \mathcal{D}] = \alpha_n/(\alpha_n{+}\beta_n) = \mathbf{0.8286}$. It *numerically* equals the posterior mean, but the two are conceptually distinct — one is about a future observation, the other summarises a parameter distribution. The coincidence is a property of the linear expectation and vanishes for anything non-linear: the probability that the next *two* attempts both succeed is $\mathbb{E}[\theta^2] = 0.6879 > (\mathbb{E}[\theta])^2 = 0.6865$ by Jensen's inequality, strictly above the plug-in answer.

(2) 95% equal-tailed credible interval: [**0.7512**, **0.8941**] from the posterior quantiles, verified by $F(\text{lo}) = 0.025$, $F(\text{hi}) = 0.975$.

(3) Threshold mass: $P(\theta > 0.8 \mid \mathcal{D}) = 1 - F(0.8) = \mathbf{0.7883}$, confirmed two ways — numerical integration gives 0.78826666 against SciPy's 0.78826666, and 200,000 posterior draws (seed `20260910`) give 0.78688, within 0.5 standard errors of the analytic value.

Decision: the rule is NOT met. It requires $P(\theta > 0.8 \mid \mathcal{D}) \geq 0.95$; the evidence gives 0.7883, a shortfall of 0.1617. At the observed success rate, roughly 400–500 further attempts would be needed to satisfy it.

Why a point estimate above 0.8 is not sufficient. Both ML (0.8500) and the posterior mean (0.8286) lie *above* the threshold, yet the rule still fails. A point estimate discards the posterior's width. With 100 attempts the posterior retains real spread (sd 0.0366) and 21.2% of its mass sits below 0.8; the credible interval's lower bound, 0.7512, is below the threshold, so $\theta < 0.8$ has not been ruled out. The rule is deliberately written about the whole distribution because the cost of an unreliable gripper is incurred when θ is genuinely low, not when the average looks acceptable. Passing requires the posterior to be *concentrated* above 0.8.

Why a PDF value at 0.8 is not this probability. $f(0.8) = 7.26$ is a density — a probability-per-unit-θ — and cannot be compared with 0.95 at all; this posterior peaks at 10.96, well above 1. A single point has probability exactly zero for a continuous parameter. The rule needs the *area* $\int_{0.8}^{1} f(\theta \mid \mathcal{D})\, d\theta$. The notebook verifies the relationship directly: $P(0.8 \pm 0.0005) = 0.0072565$ against $f(0.8) \times 0.001 = 0.0072556$, which is what a density

means. B3_decision.json, B3_mc_verification.png

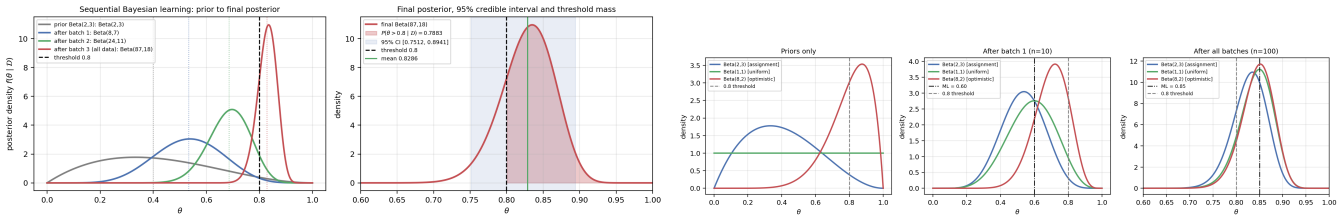


Figure 2: Left: sequential learning, and the final posterior with its 95% interval and the shaded threshold mass $P(\theta > 0.8 \mid \mathcal{D}) = 0.7883$ — visibly short of 0.95 even though the mean exceeds 0.8. **Right:** three priors after 0, 10 and 100 attempts; the posteriors converge as data accumulate.

B4. The role of the prior

Table 4: Prior sensitivity. After 10 attempts the priors disagree materially; after 100 they nearly agree. B4_prior_sensitivity.csv

	After batch 1 (n=10, ML = 0.600)			After all batches (n=100, ML = 0.850)			
Prior	Post. mean	MAP	$P(\theta{>}0.8)$	Post. mean	MAP	$P(\theta{>}0.8)$	Rule met?
Beta(2, 3) assignment	0.5333	0.5385	0.0116	0.8286	0.8350	0.7883	No
Beta(1, 1) uniform	0.5833	0.6000	0.0504	0.8431	0.8500	0.8811	No
Beta(8, 2) optimistic	0.7000	0.7222	0.1631	0.8455	0.8519	0.9009	No

After batch 1 the priors are worth 5, 2 and 10 pseudo-attempts against only 10 real ones, so the posterior means span 0.533–0.700 and the threshold probabilities differ by a factor of 14 (0.012 to 0.163). After 100 attempts the likelihood dominates: means collapse into 0.829–0.846, a spread of 0.017, all near ML = 0.850. The threshold probability stays more prior-sensitive than the mean (0.788–0.901) because it depends on the posterior *tail* rather than its centre — worth noting here, since a decision stated as a tail probability is more fragile under prior choice than one stated as a point estimate. The rule fails under all three priors, so that conclusion is robust; had it flipped, the honest reading would be that the data are too weak to support it.

ML and MAP under a uniform prior. With Beta(1, 1) the prior is constant, so it contributes no θ-dependence and maximising the posterior *is* maximising the likelihood: $\hat{\theta}_{\text{MAP}} = \frac{(1+s)-1}{(1+s)+(1+n-s)-2} = \frac{s}{n} = \hat{\theta}_{\text{ML}}$, confirmed numerically at both stages (0.600 and 0.850).

Why that does not make the Bayesian answer identical to ML. The coincidence concerns only the single highest point of the density. Everything else differs. The posterior *mean* under Beta(1, 1) is $(1+s)/(2+n) = 0.8431$ — Laplace smoothing, not 0.850 — and it is the predictive probability for the next attempt, so even the flat-prior Bayesian *prediction* differs from the ML plug-in. More fundamentally, ML returns a number while the posterior returns a distribution, from which credible intervals and $P(\theta > 0.8 \mid \mathcal{D})$ follow; ML has no mechanism to produce the quantity the deployment rule asks for. The gap matters most where decisions are riskiest: with 5 successes in 5 attempts, ML and MAP both give $\hat{\theta} = 1.0$ — implying the next attempt *cannot* fail — while the flat-prior predictive gives 0.857 with interval $[0.541, 0.996]$. Optimising a point and carrying a distribution are different operations; only the second propagates uncertainty into the decision.

Part C (option C2): animal-drawing generation with flow matching

C1. Method, and the two research questions

Goal and data. A class-conditional generative model over 28×28 grayscale line drawings: given a requested category $y \in \{\text{cat}, \text{dog}, \text{fish}\}$ and fresh Gaussian noise, produce a new drawing of that category. Data: Google Quick, Draw! `numpy_bitmap` files (CC BY 4.0, Google Creative Lab), 4,000 original drawings per category.

Straight-line flow matching. For noise $x_0 \sim \mathcal{N}(0, I)$ and a real drawing $x_1 \sim p_{\text{data}}$, define the interpolation and its *constant* velocity

$$x_t = (1-t)x_0 + t\,x_1, \qquad u_t = \frac{dx_t}{dt} = x_1 - x_0, \qquad t \sim \mathcal{U}[0,1],$$

and regress it with a time- and class-conditional network: $\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,x_0,(x_1,y)} \left\| v_\theta(t, x_t, y) - (x_1 - x_0) \right\|^2$. Because the path is a straight line, the target is available in closed form and *no ODE is simulated during training.* Generation integrates the learned ODE $dx/dt = v_\theta(t, x, y)$ from t=0 to t=1.

Training (targets known in closed form): $x_0 \sim \mathcal{N}(0,I)$, $x_1 \sim p_{\text{data}}$, $t \sim \mathcal{U}[0,1]$ $\rightarrow$ $x_t = (1-t)x_0 + t x_1$ $\rightarrow$ $v_\theta(t, x_t, y)$ $\xrightarrow{\text{MSE vs } x_1 - x_0}$ ∇
Generation (integrate forward): $x_0 \sim \mathcal{N}(0,I)$ $\rightarrow$ $\left[x \leftarrow x + \Delta t \cdot v_\theta(t, x, y) \right] \times n_{\text{steps}}$, $t: 0 \rightarrow 1$ $\rightarrow$ $x(1) =$ drawing

A single x_t could have arisen from many (x_0, x_1) pairs, so what does the regression learn? The MSE-optimal prediction is the *conditional mean* $\mathbb{E}[x_1 - x_0 \mid x_t, y]$, and it is a theorem of flow matching that this marginal velocity field transports $\mathcal{N}(0, I)$ to p_{data} exactly. The network learns the average direction of travel, not the particular pair.

Link to the unit. The same three-part pattern as Parts A and B. *Known structure:* the noise distribution and the interpolation path, both chosen by me, not learned. *Unknown component learned from data:* the velocity field — which, exactly like the quantities estimated in A and B, is a conditional expectation. *Generation by sampling:* integrate the ODE from a fresh x_0. The analogy with A4 is precise: the interpolation defines a *known* forward corruption $x_1 \to x_t$, but the forward process alone does not determine the reverse dynamics, because $\mathbb{E}[x_1 - x_0 \mid x_t]$ depends on p_{data} — just as fixing downstream evidence in a forward sampler does not give the posterior without a correction.

Q1 (training factor). Does longer training improve recognisability, and at what cost to diversity? *Predicted:* sharp gain from epoch 5 to 15, then flat; loss keeps falling; diversity constant. **Q2 (inference factor).** How few ODE steps suffice, and does midpoint beat Euler at matched network evaluations (NFE)? *Predicted:* few steps suffice, saturating by 10–25; at matched NFE midpoint gains little, because straight-trained paths have little curvature for a second-order solver to exploit.

C2. Implementation, and three key code sections

Table 5: Baseline configuration. Trained entirely from scratch — no pretrained weights, nothing frozen. `C_reproduction_record.json`

Data	4,000 orig. drawings/class × 3	Architecture	`VelocityUNet` 28→14→7, ch. 64→128
Split	9,600 train / 2,400 held out (orig.)	Trainable params	1,838,081 (all of them)
Preprocessing	$[0, 255] \to [-1, 1]$, no resize	Conditioning	sinusoidal t + class embedding, per block
Augmentation	random h-flip, train split only	Optimiser	AdamW, lr 2×10^{-4}, cosine, wd 10^{-4}
Filter	ink fraction $\in [0.05, 0.55]$ (8/20,700 rejected)	Batch / epochs	128 / 60, grad-norm clip 1.0
Seeds	split+train `20260910`; 32 generation seeds	Hardware / time	RTX 3060 (12 GB), **6.39 min**

(a) Training examples and targets — `flow_matching_batch` (`flow_model.py`), called in `train()`. *In:* clean drawings `x1` $(B, 1, 28, 28)$. *Out:* `(x_t, t, target)`. It draws Gaussian `x0` of the same shape and one uniform `t` per image, reshaped to $(B, 1, 1, 1)$ so it broadcasts across all pixels of that image (a per-image scalar, not per-pixel), forms `x_t = (1-t)*x0 + t*x1`, and returns `x1 - x0`. Each example is a fresh (x_0, t) pair, so a drawing is seen at many noise levels.

(b) Loss and updates — inner loop of `train()` (`train_flow.py`). *In:* a batch `(x1, y)`, model, optimiser. *Out:* one gradient step. It applies the flip augmentation (train split only), builds the triple, and computes `loss = mean((model(t,x_t,y) - target)**2)` — this *is* $\mathcal{L}_{\text{CFM}}$, with the expectation over t, x_0 realised by the random batch. Then `zero_grad`→`backward`→clip→`step`. The whole generative model reduces to a regression. `eval_loss` measures held-out loss over a *fixed* grid of t so checkpoints are comparable rather than differing by sampling noise.

(c) Generation — `sample_ode` (`flow_model.py`). *In:* model, count, labels, `n_steps`, `method`, `seed`. *Out:* images, **NFE**, optional trajectory. It seeds a generator, draws x_0 — *the only randomness* — and steps $t : 0 \to 1$. Euler uses one network evaluation per step; midpoint two. NFE is returned explicitly because comparing solvers at matched *step count* would silently give midpoint double the compute.

Verification before training. I asserted the shapes match, that *all* parameter tensors receive non-zero gradient, and specifically that `class_embed.weight.grad` is non-zero — a dead conditioning path fails silently, since the loss still falls while every class generates the same thing. Endpoint checks confirmed $\max |x_t - x_0|$ at t=0 and $\max |x_t - x_1|$ at t=1 are $\sim 10^{-7}$.

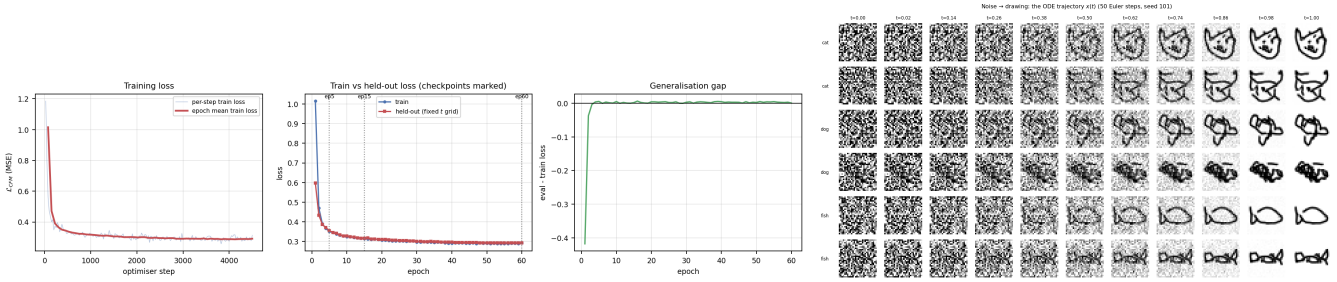


Figure 3: Left: training and held-out loss (checkpoints marked). Held-out loss never exceeds train loss by more than 0.006 — no overfitting at this size. **Right:** the ODE trajectory $x(t)$ from noise to drawing; broad structure appears by $t \approx 0.3$–0.5 and later steps sharpen strokes, consistent with near-straight paths. `C_training_curves.png`, `C_trajectory_noise_to_drawing.png`

C3–C4. Controlled comparisons and evaluation

Protocol. A predetermined list of **32 seeds**, fixed once and reused in *every* comparison; each seed gives one sample per class, so **96 outputs per configuration** (32 per class, against the required minimum of 16 total and 4 per class). I exceed the minimum deliberately: a first run with 8 seeds (24 outputs) returned an identical 0.75 for all three checkpoints, a discretisation artefact rather than a finding. One factor changes at a time; data and evaluation inputs are fixed. **Quality measure:** *recognisability* — the fraction of a configuration's full 96-sample set that an independent CNN assigns to the *requested* class. That CNN is trained *only on real held-out drawings*, data the generator never saw, and it scores **0.8375** on real held-out drawings — so 0.8375, not 1.0, is the ceiling.

Table 6: Both comparisons. Binomial standard error is ±0.041 throughout (n=96), so differences below ∼0.09 are not resolvable. `C_all_results.csv`

	Config	Held-out loss	Recog. (±0.041)	Confidence	Diversity	NFE/img	ms/img
training	epoch 5	0.3557	0.781	0.637	22.6	50	16.3
	epoch 15	0.3164	0.771	0.672	22.1	50	16.3
	epoch 60 (baseline)	0.2922	**0.802**	**0.704**	22.5	50	16.3
	epoch 60, *quarter* data	0.3297	0.760	—	22.1	50	16.3
inference	Euler 2 steps	—	0.781	0.630	**14.9**	2	0.76
	Euler 5 steps	—	0.792	0.703	20.6	5	1.74
	Euler 10 steps	—	0.792	0.706	21.7	10	3.35
	Euler 50 steps	—	0.802	0.704	22.5	50	16.3
	Euler 100 steps	—	0.792	0.702	22.6	100	32.1
	Midpoint 5 steps	—	0.802	0.707	22.7	10	3.32



Figure 4: Matched grids, identical seeds down the columns, first 12 of 32 seeds shown; failures included. **Left (Q1):** only the checkpoint differs. Visual quality clearly improves — epoch-5 cats are scribbly, epoch-60 cats have clean ears and faces, epoch-60 fish are crisp — even though the recognisability *metric* moved within noise. **Right (Q2):** only the ODE step count differs; the 2-step row is visibly faint and washed out, while 10 and 50 steps are near-indistinguishable. Complete 96-sample sets for all 14 configurations, including `.npz` arrays, are in `results/partC/sample_grids/`.

C5. Interpretation

Q1: my prediction was wrong, and how it is wrong is the main result. Held-out loss fell monotonically by 18% (0.356→0.292) and mean confidence rose steadily (0.637→0.704), but *recognisability barely moved*: 0.781→0.771→0.802, a change of half a standard error. I cannot claim a real accuracy gain from training 12× longer. Three conclusions follow. *First*, this is a direct quantitative demonstration that **training loss is not a measure of generated-image quality** — loss improved 18% while the sample-level score stayed within noise, because the CFM loss averages velocity error over all t and most residual reduction happens at noise levels that barely affect where the trajectory lands. *Second*, confidence was the more sensitive instrument: the thresholded arg-max decision discards information that longer training genuinely improved, so reporting accuracy alone would have hidden a real effect — while reporting it *as if* it showed one would have invented an effect. *Third*, the task saturates almost immediately: by epoch 5 (32 seconds) samples already score 0.781 against a ceiling of 0.8375, leaving little headroom. Crucially, the grids in Fig. 4 show that visual quality *does* improve substantially — so the honest conclusion is that my *metric* was too coarse for this factor, not that training does nothing. Per-class results locate the difficulty: `fish` scores 0.94–0.97 at every checkpoint while `dog` scores 0.53–0.66, mirroring the classifier's own real-data ceilings (fish 0.916, cat 0.821, dog 0.781) — dogs are the hardest class in this dataset for evaluator and generator alike, so this reflects the data, not broken conditioning. The *dataset-size* comparison is the training factor that does move: quartering the data at identical optimisation budget raises held-out loss 0.292→0.330 and lowers recognisability 0.802→0.760. At this model size the binding constraint is data, not steps.

Q2: both halves of the prediction hold, and saturation is more extreme than predicted. Recognisability is flat from 2 to 100 Euler steps — all within one standard error. What *does* change at 2 steps is **diversity**, collapsing from ∼22.5 to **14.9** (a 34% drop, far outside the variability elsewhere), with confidence also falling to 0.630. The 2-step failure mode is therefore not "wrong class" but "under-integrated": samples land in roughly the right region yet are washed-out and mutually similar, exactly as Fig. 4 (right) shows. Beyond ∼5 steps extra steps buy nothing measurable while cost grows linearly — 0.76 ms to 32.1 ms per image from 2 to 100 steps, a 42× increase for no measurable gain. This is flow matching's practical appeal over DDPM sampling: straight-line targets give near-straight trajectories, so a handful of steps suffices where diffusion typically needs hundreds. At **matched NFE** the Euler/midpoint differences are +0.010, −0.010 and 0.000, each far below the ±0.058 standard error of the difference, so no configuration is distinguishable — matching the geometric argument that a second-order solver's advantage scales with curvature. Midpoint's only visible edge is at the very lowest budget: at NFE 4 it recovers

diversity 23.7 against Euler's 14.9 at NFE 2. C_comparison2_matched_nfe.csv

Why generation is random although the solver is deterministic. Both solvers are deterministic maps — fixing x_0 fixes the output exactly, which is what makes the 32-seed list reproducible. I verified this rather than assuming it, and the check caught something worth recording: with default settings two runs of the *same* seed differed by $\sim 7 \times 10^{-4}$, because cuDNN is free to choose non-deterministic convolution algorithms and TF32 reductions. The mathematics was never the problem — x_0 was bitwise identical — but the claim "the same seed reproduces the same image" was only true to plotting precision until I pinned the backend (`enable_deterministic()` in `flow_model.py`), after which same-seed runs are bitwise equal and the notebook asserts it. All randomness enters through the *initial condition* $x_0 \sim \mathcal{N}(0, I)$; the ODE deterministically transports the noise distribution onto the data distribution. Variety comes from where you start, not from noise injected en route — the structural difference from a stochastic diffusion sampler.

Memorisation. Mean L_2 distance from each of the 96 generated samples to its nearest *training* drawing is **16.62**, against **15.69** for 200 real *held-out* drawings measured the same way. Generated samples are thus *further* from the training set than genuinely unseen real drawings are — evidence against wholesale copying. Inspecting the six closest pairs, they share only coarse layout (a body outline, a tail direction), not stroke-for-stroke identity. Resemblance alone is not proof: with 3,200 training drawings per class of simple doodles, any plausible new cat *must* resemble some training cat, so the distance *distribution* is the meaningful comparison, not any single pair. C_memorisation_distances.csv, C_memorisation_probe.png

Limitations of the measure, and of the study. (i) The ceiling is 0.8375, not 1.0, because Quick, Draw! doodles are genuinely ambiguous; epoch 60 reaches 96% of it. (ii) The classifier is an imperfect proxy with its own biases — a high score is necessary but not sufficient for "a human would call this a cat". (iii) **It measures conditioning, not fidelity, and this bit me**: the 2-step samples scored 0.781, statistically indistinguishable from 50 steps, despite being obviously worse and showing a 34% diversity collapse. Class accuracy was nearly blind to the largest quality difference in the study. (iv) At n=96 the standard error is 0.041, so sub-0.09 differences are unresolvable; only two effects in this study exceed noise — the 2-step diversity collapse and the quarter-data loss degradation. The rest are null results, and detecting smaller effects would need several hundred samples per configuration (affordable at 17 ms/image, and the first thing I would change). (v) L_2 nearest-neighbour distance is a weak perceptual metric: adequate for near-duplicate detection, blind to a semantically memorised drawing shifted by a few pixels. (vi) 28×28 line drawings are the easy end of image generation; nothing here shows the approach scales to photographs. No FID or human study — both need far more samples or a pretrained feature extractor, and I report only measures I can define and defend at this scale.

Conclusion: inference → learning → generation

Part A took a *fully known* model and did inference by sampling, where the entire difficulty was conditioning on evidence correctly — overwriting downstream evidence returned the prior (0.0799 vs $q_2 = 0.2933$), and rejection and likelihood weighting are the two valid corrections. Part B kept the structure known but made *one scalar* unknown, and learning a posterior over it changed the decision: ML = 0.850 sits above the threshold, yet $P(\theta > 0.8 \mid \mathcal{D}) = 0.788$ fails the 0.95 rule, because a point estimate discards the width that the rule is explicitly about. Part C makes the unknown a *function*: the noise distribution and interpolation path are chosen by me, and the velocity field — a conditional expectation, exactly like the quantities estimated in A and B — is learned from data; generation is then sampling in the same sense, pushing a draw from a known distribution through a learned map. The through-line is that a generative model is not a different *kind* of object from a Bayesian network: it is the same factorise–learn–sample pattern with a neural network standing in for a CPT, and the same warning holds at every scale — a known forward process never hands you the reverse one for free. Part C also delivered the methodological lesson the brief anticipates: an 18% improvement in training loss produced no measurable change in sample quality, so the measure you choose determines what you are able to conclude.

AI-use statement

I used Claude (Anthropic), model Opus 5, via the Claude Code CLI on 9–10 September 2026, for explanation of flow matching for images, code scaffolding for the U-Net and sampler, notebook structure and debugging. All mathematical reasoning, experimental design, verification and claims are my own, and every number reported was produced by code I ran on my own hardware. Five interactions are documented in `AI_USE_LOG.md` with prompt, response excerpt, my check and the resulting decision, including one independently verified correctness check (I re-derived Part A's exact values with a separate brute-force enumeration agreeing to 10 decimal places, and added a transposed-CPT test the assistant had not suggested, showing the normalisation check is weaker than it looks) and one prediction-versus-observation comparison (the Euler/midpoint expectation above, which the data contradicted). I also record one suggestion I **rejected**: proposed ink-filter thresholds of $[0.02, 0.60]$ rejected *nothing* when I measured the actual distribution, so I recalibrated to $[0.05, 0.55]$ rather than disclose a filter that did not filter. The log's summary table lists all 14 independent checks I ran. No result, runtime, citation or figure in this submission was fabricated.

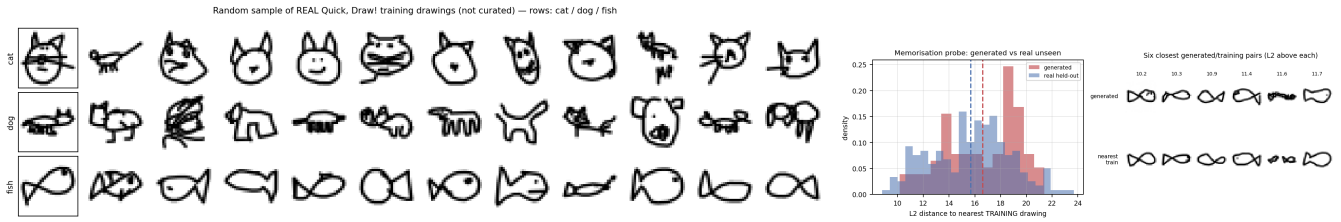


Figure 5: Left: a random (not curated) sample of the real Quick, Draw! *training* drawings, inspected before training. Several are ambiguous even to a human — these are 20-second public doodles — which is what sets the 0.8375 recognisability ceiling rather than 1.0. **Right:** the memorisation probe. Generated samples sit at mean L_2 16.62 from their nearest training drawing versus 15.69 for real held-out drawings, and even the six closest pairs share only coarse layout, not strokes. C_real_data_grid.png, C_memorisation_probe.png

References

1. COMP6010 teaching materials (2026): *Bayesian Networks I*; *Bayesian Networks 2 – Inference*; *Learning Probabilistic Models*; *Diffusion and Flow Matching Models*; continuous-probability notes; Practicals 1–4, 6 and 7. Used as the primary source for all three parts.
2. Google Creative Lab. *The Quick, Draw! Dataset* (`numpy_bitmap` format), CC BY 4.0. `https://github.com/googlecreativelab/quickdraw-dataset`. Categories `cat`, `dog`, `fish` retrieved 10 September 2026; SHA-256 checksums in `C_data_manifest.json`.
3. SciPy documentation, `scipy.stats.beta` (v1.15.3) — Beta densities, CDF/survival functions, quantiles and sampling. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.beta.html`
4. Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., Le, M. (2023). Flow Matching for Generative Modelling. *ICLR*. `https://arxiv.org/abs/2210.02747`
5. Liu, X., Gong, C., Liu, Q. (2023). Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. *ICLR*. `https://arxiv.org/abs/2209.03003`
6. Meta AI. *flow_matching* library and example notebooks (adapted conceptually for the 2-D starting point in Practical 7). `https://github.com/facebookresearch/flow_matching`
7. Ho, J., Jain, A., Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *NeurIPS*. `https://arxiv.org/abs/2006.11239` — used for the contrast with stochastic samplers in §C5.
8. Paszke, A. et al. (2019). PyTorch (v2.10.0+cu128). `https://pytorch.org`
9. Kish, L. (1965). *Survey Sampling*. Wiley — effective sample size used in §A4.